SESIÓN 2 · SEMANA 2 · UNIDAD 1

Evolución del Software y Mantenimiento

Leyes de Lehman, deuda técnica y los cuatro tipos de mantenimiento

Checkpoint del proyecto: Actividad 1 (15%)

Agenda de hoy (120 minutos)

10 min	Bienvenida y objetivos de la sesión
15 min	Checkpoint: Actividad 1 - qué se evalúa
50 min	Leyes de Lehman, deuda técnica y tipos de mantenimiento
20 min	Casos de la industria
15 min	Taller: diagnóstico exprés en equipos
10 min	Cierre y próximos pasos

Punto de control del proyecto

Análisis de evolución, deuda técnica y mantenimiento

15%

de la nota final

1.0	Definición del reto: claro, pertinente y alineado con el sistema
1.0	Caracterización completa del sistema (componentes, usuarios, contexto)
1.0	Identificación y justificación de los problemas del sistema
1.0	Análisis profundo de evolución y deuda técnica
1.0	Presentación clara, con sustento teórico y normas APA

La naturaleza evolutiva del software

1. El software que opera en un dominio real no es estático: se deteriora por los efectos colaterales de los cambios, no por desgaste físico.

2. Lehman clasificó estos sistemas como Tipo E: deben adaptarse continuamente o se vuelven progresivamente menos satisfactorios.

3. El mantenimiento no es una fase posterior al desarrollo: es el desarrollo continuo bajo otra forma.

60-70%

de la capacidad de ingeniería se destina a mantener sistemas existentes

Pressman (2010)

Ejemplo: SIGECLIN, el sistema de Vitalis Salud, lleva más de ocho años en producción sin un rediseño de fondo. Así se ve, en la práctica, un sistema Tipo E.

Cambio continuo y complejidad creciente

1. Ley del Cambio Continuo: un sistema que no evoluciona pierde utilidad, aunque no tenga errores visibles.

2. Ley de la Complejidad Creciente: cada cambio sin simplificación paralela encarece el siguiente.

3. Tres síntomas de entropía: dilatación de tiempos, inestabilidad operativa y silos de conocimiento.

Principio de Incertidumbre del Software

Un sistema que ha funcionado bien puede fallar de forma súbita, no por cambios en su código, sino porque los supuestos originales dejaron de ser válidos. (Lehman, 1996)

Ejemplo: en el cohete Ariane 5 (1996), el código reutilizado del Ariane 4 funcionaba bien en su contexto original, pero los supuestos de trayectoria dejaron de ser válidos con el nuevo cohete.

Los cuatro tipos de mantenimiento

No son categorías académicas: son un indicador de cómo un equipo gestiona su software

C Correctivo

Reparar defectos detectados en producción. Necesario, pero insuficiente si es el único tipo que se practica.

Ejemplo: en Banco Meridiano, corregir en caliente un error de redondeo en el cálculo de comisiones reportado por un cliente.

A Adaptativo

Ajustar el sistema cuando cambia el entorno: sistema operativo, API externa, nueva regulación.

Ejemplo: en Vitalis Salud, ajustar SIGECLIN para cumplir una nueva resolución del Ministerio de Salud sobre historias clínicas.

P Perfectivo

Mejorar funcionalidad o rendimiento sin que exista un defecto. El más postergado bajo presión de entrega.

Ejemplo: Etsy rediseñó su proceso de checkout para reducir tiempos de carga, sin que existiera un defecto reportado.

V Preventivo

Corregir fallas latentes antes de que se manifiesten. Requiere anticipación y tiempo disponible.

Ejemplo: equipos de SRE ejecutan análisis estático y pruebas de carga antes de cada despliegue, para detectar fallas antes de producción.

El costo que se paga después

1. La deuda técnica rara vez nace de la negligencia: nace de la velocidad y la presión por entregar.

2. No toda deuda es irresponsable: puede ser deliberada, como validar un MVP. El problema es la deuda que nadie registra ni cuantifica.

3. Como cualquier deuda financiera, tiene intereses: bugs, lentitud y rigidez que se acumulan con el tiempo.

20-40x

el costo de mantener una línea de código puede superar el de su desarrollo original

Pressman (2010)

Ejemplo: en OP24, el sistema de Banco Meridiano, una dependencia desactualizada que nunca se retiró provocó una falla silenciosa en el cálculo del puntaje de fraude.

Cuando la deuda no gestionada se cobra

2020

NJ Unemployment / COBOL

Un incremento masivo de solicitudes expuso la fragilidad de mainframes con décadas de antigüedad, obligando a buscar programadores COBOL de emergencia.

El estado había postergado la modernización del sistema durante décadas. Cuando llegó la crisis, la deuda técnica se cobró en tiempo real, mientras miles de personas esperaban su pago de desempleo.

2012

Knight Capital

Una configuración de despliegue defectuosa activó código obsoleto, generando pérdidas de 460 millones de dólares en 45 minutos.

La empresa quebró en menos de una hora. No fue un solo error: fue código obsoleto que debió eliminarse años antes y nunca se retiró del todo. Es el ejemplo más citado del costo de no cerrar un ciclo de mantenimiento.

1996

Ariane 5

La reutilización de código del Ariane 4 sin validar los nuevos valores de trayectoria provocó la pérdida del cohete a los 37 segundos del lanzamiento.

El equipo asumió, sin comprobarlo, que el código que había funcionado bien en el Ariane 4 seguiría siendo válido en el nuevo cohete. Ese supuesto no verificado costó el lanzamiento completo.

Taller: diagnóstico exprés (15 minutos, en equipo)

1 **Elijan un módulo crítico**

Del sistema que seleccionaron en la Actividad 0.

2 **Identifiquen 2 señales de deuda técnica**

Evidencia concreta, no una impresión general.

3 **Clasifiquen el mantenimiento pendiente**

Ubíquenlo en las 4 categorías vistas hoy.

4 **Compartan un hallazgo**

Un integrante lo socializa con el resto del curso.

Para reflexionar y debatir

R

Reflexionar: piensa en un sistema con el que hayas trabajado. ¿Qué tipo de mantenimiento consumía la mayor parte del tiempo del equipo? ¿Qué dice esa distribución sobre las prioridades reales de la organización?

D

Debatir: ¿la deuda técnica es inevitable en entornos de alta presión, o no gestionarla simplemente difiere costos mayores? ¿Qué posición te parece más defendible en tu contexto?

Lecturas para la Sesión 3 (Unidad 2)

OBLIGATORIAS

Berczuk, S., & Appleton, B. (2002). Software Configuration Management Patterns (cap. 1-3). Addison-Wesley.

IEEE. (2017). IEEE 828 - Configuration Management (secciones 1-4).

Kim, G., Willis, J., Debois, P., & Humble, J. (2016). The DevOps Handbook (cap. 4: Version Control). IT Revolution Press.

COMPLEMENTARIAS

Atlassian. (2023). Git Tutorials. atlassian.com/git/tutorials

Git SCM. (2024). Git Documentation. git-scm.com/docs

Antes de la Sesión 3

- Completar el documento de diagnóstico de la Actividad 1
- Registrar al menos 3 commits incrementales en docs/unidad-1/
- Leer los materiales de la Unidad 2 (diapositiva anterior)
- Traer dudas sobre configuration items y baselines para la próxima sesión

Nos vemos en la Sesión 3 · Unidad 2 · Gestión de configuración